

多模型管理与快速切换方案

核心思路：每个改进只训练一次，checkpoint 永久保留在 `model_zoo/` 下，评测时通过参数切换模型，无需重训。
当前执行路线详见 `action_plan.md`。

目录

- [最重要：代码版本与 checkpoint 的绑定关系](#)
- [各阶段代码改动与兼容性分析](#)
- [当前计划的模型列表](#)
- [目录结构约定](#)
- [训练后保存 checkpoint](#)
- [单模型评测（参数化调用）](#)
- [批量评测](#)
- [结果对比](#)
- [完整工作流示例](#)
- [磁盘空间参考](#)

最重要：代码版本与 checkpoint 的绑定关系

不同阶段的改进会修改模型架构或配置，导致旧代码无法加载新 checkpoint，反之亦然。必须用对应的代码版本加载对应的 checkpoint，否则报错或静默加载错误权重。

checkpoint	对应代码分支	不兼容的原因	能否用 baseline 代码加载？
<code>uninavid_baseline</code>	<code>main</code>	—	✅ 是
<code>uninavid_s1_llama31_curriculum</code>	<code>feat/llama31</code>	LLM 从 LLaMA-2 换为 LLaMA-3.1, tokenizer 和模型配置不同	❌ 否
<code>uninavid_s2_dpo</code>	<code>feat/llama31</code>	与 S1 架构完全一致, 只是权重不同	❌ 否 (同 S1)
<code>uninavid_s3_siglip</code>	<code>feat/siglip</code>	视觉编码器变更, <code>mm_hidden_size</code> 从 1408 变为 1152, projector 结构不同	❌ 否

管理方式：用 **Git** 分支隔离代码版本

```
1 | main                                ← baseline 代码, 加载 uninavid_baseline
2 |   └─ feat/llama31                  ← S1/S2 代码, 加载 s1 / s2 checkpoint
3 |     └─ feat/siglip                 ← S3 代码, 在 feat/llama31 基础上修改, 加载 s3 checkpoint
```

评测时切换分支即切换对应代码, 再指定 `MODEL_PATH` 即可加载对应 checkpoint:

```
1 | # 评测 baseline
2 | git checkout main
3 | bash eval.sh --model uninavid_baseline --benchmark r2r
4 |
5 | # 评测第一/二阶段
6 | git checkout feat/llama31
7 | bash eval.sh --model uninavid_s1_llama31_curriculum --benchmark r2r
8 | bash eval.sh --model uninavid_s2_dpo --benchmark r2r
9 |
10 | # 评测第三阶段
11 | git checkout feat/siglip
12 | bash eval.sh --model uninavid_s3_siglip --benchmark r2r
```

ToMe (第零阶段) 通过 flag 控制, `--use-tome` 在任意分支上均可选择性启用, 不影响 checkpoint 兼容性。

各阶段代码改动与兼容性分析

第零阶段：改进 10 (ToMe)

修改文件: `uninavid/model/multimodal_encoder/eva_vit.py`

改动内容: 在 ViT Transformer Block 的注意力计算前后插入 ToMe merge / unmerge 步骤。

兼容性:

- ✓ 不改变 checkpoint 格式, 权重完全兼容
- ✓ 通过 `use_tome: bool` 参数控制是否启用, 可随时开关
- ✓ 所有分支均可使用, 不影响其他阶段

注意: ToMe 插入后模型的中间输出 shape 会变化, 但最终输出 shape 不变 (unmerge 还原), checkpoint 加载无影响。

第一阶段：改进 1 + 6 (LLaMA-3.1-8B + 课程学习)

改进 1 代码改动

修改文件:

- `uninavid/model/language_model/llava_llama_vid.py`: 确认 `LlamaForCausalLM` 兼容性 (LLaMA-3.1 架构与 LLaMA-2 高度一致, 通常无需改动)
- `scripts/uninavid_stage_1.sh`: `PREV_MODEL` 路径改为 LLaMA-3.1-8B 权重路径
- `requirements`: `transformers>=4.40`

checkpoint 配置变化:

```
1 config.json 中:
2   "_name_or_path": "meta-llama/Meta-Llama-3.1-8B-Instruct"    ← 变化
3   "vocab_size": 128256    ← 变化 (LLaMA-3 词表比 LLaMA-2 大)
4   "bos_token_id": 128000 ← 变化
5   "eos_token_id": 128001 ← 变化
6
7 tokenizer_config.json:
8   使用 LLaMA-3 tokenizer (tiktoken)    ← 完全不同, 不可混用
```

不兼容原因: 词表大小 (32000 → 128256) 和 tokenizer 格式完全不同, 用 baseline 代码加载会导致 embedding 维度不匹配。

分支操作:

```
1 git checkout -b feat/llama31
2 # 修改上述文件后提交
3 git add uninavid/model/language_model/llava_llama_vid.py
4 git add scripts/uninavid_stage_1.sh
5 git commit -m "feat: replace LLaMA-2 base with LLaMA-3.1-8B"
```

改进 6 代码改动

修改文件: uninavid/train/train.py 中的 LazySupervisedDataset

改动内容: 在数据集初始化时, 按路径难度字段 (步数、转弯次数) 对样本排序, 训练前期用简单样本, 后期逐步引入复杂样本。

兼容性:

- ☒ 纯训练逻辑改动, 不影响模型结构, checkpoint 格式与改进 1 完全相同
- ☒ 训练完成后 checkpoint 可与 S1 的代码/评测脚本直接互换

```
1 # 在 feat/llama31 分支上追加提交
2 git add uninavid/train/train.py
3 git commit -m "feat: add curriculum learning to LazySupervisedDataset"
```

第二阶段: 改进 2 (DPO)

修改文件: 新增 uninavid/train/train_dpo.py (DPO 训练入口)

改动内容:

- 引入 trl.DPOTrainer, 接受 {"prompt", "chosen", "rejected"} 三元组数据
- 在 S1 checkpoint 基础上继续训练, 不修改模型结构

兼容性:

- ☒ 模型架构与 S1 完全一致, config.json 无变化
- ☒ S2 checkpoint 与 S1 代码完全兼容, 可用同一套评测代码加载
- ☒ 在 feat/llama31 分支上追加文件即可, 无需新分支

checkpoint 关系:

```
1 uninavid_s1_llama31_curriculum —DPO fine-tune—> uninavid_s2_dpo
2      (起点)                                (结果)
3      同一架构, 只有权重数值不同
```

```
1 # 仍在 feat/llama31 分支
2 git add uninavid/train/train_dpo.py
3 git commit -m "feat: add DPO training script"
```

第三阶段：改进 8（SigLIP 编码器）

修改文件：

- 新增 `uninavid/model/multimodal_encoder/siglip_encoder.py`
- `uninavid/model/multimodal_encoder/builder.py`：添加 SigLIP 路由分支
- `uninavid/model/uninavid_arch.py`： `mm_hidden_size` 读取逻辑兼容 1152

checkpoint 配置变化：

```
1 config.json 中:
2   "mm_vision_tower": "google/siglip-so400m-patch14-384"  ← 变化
3   "mm_hidden_size": 1152                                ← 变化 (原 1408)
4
5 projector 权重维度:
6   原: Linear(1408, 4096)
7   新: Linear(1152, 4096)  ← 形状不同, 不可共用
```

不兼容原因：projector 的 Linear 层 shape 不同，用旧代码加载会直接报 shape mismatch 错误。

分支操作：

```
1 git checkout feat/llama31
2 git checkout -b feat/siglip
3 # 修改上述文件后提交
4 git add uninavid/model/multimodal_encoder/siglip_encoder.py
5 git add uninavid/model/multimodal_encoder/builder.py
6 git add uninavid/model/uninavid_arch.py
7 git commit -m "feat: replace EVA ViT-g with SigLIP-S0400M encoder"
```

训练方式（只需 Stage 1）：

```
1 # Stage 1: 视觉编码器 frozen, 只训练 projector (约 1-2 小时)
2 bash scripts/uninavid_stage_1_siglip.sh
3 # 起点: 使用 S2 (DPO) 的 LLM 权重 + 随机初始化的新 projector
4 # 视觉编码器: SigLIP-S0400M (frozen)
5
6 # Stage 2: 无需重跑, 直接用 Stage 1 产出的 checkpoint 做评测
```

当前计划的模型列表

阶段	checkpoint 名称	对应改进	代码分支	训练方式
baseline	uninavid_baseline	原始 Vicuna-7B	main	已有
第零阶段	无新 checkpoint	改进 10 (ToMe)	任意分支 + --use-tome	无需训练
第一阶段	uninavid_s1_llama31_curriculum	改进 1 + 6 (合并)	feat/llama31	Stage 1 + Stage 2
第二阶段	uninavid_s2_dpo	改进 2 (DPO)	feat/llama31	DPO fine-tune (小时级)
第三阶段	uninavid_s3_siglip	改进 8 (SigLIP)	feat/siglip	仅 Stage 1 (1-2 小时)
第三阶段 (可选)	uninavid_s3_siglip_dino	改进 8 + 13 (DINOv2)	feat/siglip	Stage 1 重训
长期规划	uninavid_lt_grpo	改进 18 (PPO/GRPO)	feat/grpo	在线 RL, 独立流程

目录结构约定

1	~/Uni-NaVid/	← 训练仓库
2	output/	
3	uninavid_s1_llama31_curriculum/	← 训练产出 (原始位置)
4	uninavid_s2_dpo/	
5	uninavid_s3_siglip/	
6		
7	~/NaVid-VLN-CE/	← 评测仓库
8	model_zoo/	← 所有 checkpoint 统一入口 (软链接)
9	uninavid_baseline/	← 原始模型 (已有)
10	uninavid_s1_llama31_curriculum/	← → 软链接到 Uni-NaVid/output/
11	uninavid_s2_dpo/	← → 软链接
12	uninavid_s3_siglip/	← → 软链接
13	uninavid_s3_siglip_dino/	← → 软链接
14		
15	tmp/	← 评测结果
16	results_baseline_r2r/	
17	results_baseline_tome_r2r/	← ToMe 对比
18	results_s1_llama31_curriculum_r2r/	
19	results_s2_dpo_r2r/	
20	results_s3_siglip_r2r/	

训练后保存 checkpoint

软链接到 model_zoo（推荐）

```
1 cd ~/NaVid-VLN-CE
2
3 # 第一阶段完成后
4 ln -s ~/Uni-NaVid/output/uninavid_s1_llama31_curriculum \
5     model_zoo/uninavid_s1_llama31_curriculum
6
7 # 第二阶段完成后
8 ln -s ~/Uni-NaVid/output/uninavid_s2_dpo \
9     model_zoo/uninavid_s2_dpo
10
11 # 第三阶段完成后
12 ln -s ~/Uni-NaVid/output/uninavid_s3_siglip \
13     model_zoo/uninavid_s3_siglip
```

第零阶段（ToMe）不产生新 checkpoint

```
1 # --use-tome 是运行时 flag，不改变权重，结果存到独立目录便于对比
2 bash eval.sh --model uninavid_baseline --benchmark r2r # 不启用
3 bash eval.sh --model uninavid_baseline --benchmark r2r --use-tome # 启用 ToMe
```

单模型评测（参数化调用）

必须先切换到对应代码分支，再执行评测：

```
1 # — baseline —————
2 git -C ~/Uni-NaVid checkout main
3 bash eval.sh --model uninavid_baseline --benchmark r2r
4
5 # — 第零阶段：ToMe（任意分支均可） —————
6 bash eval.sh --model uninavid_baseline --benchmark r2r --use-tome
7
8 # — 第一/二阶段（feat/llama31 分支） —————
9 git -C ~/Uni-NaVid checkout feat/llama31
10 bash eval.sh --model uninavid_s1_llama31_curriculum --benchmark r2r
11 bash eval.sh --model uninavid_s2_dpo --benchmark r2r
12
13 # — 第三阶段（feat/siglip 分支） —————
14 git -C ~/Uni-NaVid checkout feat/siglip
15 bash eval.sh --model uninavid_s3_siglip --benchmark r2r
```

`eval.sh` 待后续创建，内部自动推导 `MODEL_PATH=model_zoo/$MODEL`、`SAVE_PATH=tmp/results_${MODEL}_${BENCHMARK}`。

批量评测

由于不同 checkpoint 依赖不同代码分支，批量评测需按分支分组执行：

```
1 # 第一步：main 分支的评测
2 git -C ~/Uni-NaVid checkout main
3 bash eval.sh --model uninavid_baseline --benchmark r2r
```

```

4  bash eval.sh --model uninavid_baseline --benchmark r2r --use-tome
5
6  # 第二步: feat/llama31 分支的评测
7  git -C ~/Uni-NaVid checkout feat/llama31
8  bash eval.sh --model uninavid_s1_llama31_curriculum --benchmark r2r
9  bash eval.sh --model uninavid_s2_dpo --benchmark r2r
10
11 # 第三步: feat/siglip 分支的评测
12 git -C ~/Uni-NaVid checkout feat/siglip
13 bash eval.sh --model uninavid_s3_siglip --benchmark r2r
14
15 # 最后: 汇总对比
16 python compare_results.py --tmp-dir tmp --sort sr

```

`batch_eval.sh` 可封装上述逻辑，待后续创建。

结果对比

所有阶段评测完成后，结果均在 `tmp/` 下，随时可对比：

```

1  # 按阶段顺序对比
2  python compare_results.py \
3      --paths tmp/results_baseline_r2r \
4              tmp/results_baseline_tome_r2r \
5              tmp/results_s1_llama31_curriculum_r2r \
6              tmp/results_s2_dpo_r2r \
7              tmp/results_s3_siglip_r2r \
8      --sort sr

```

`compare_results.py` 待后续创建，详见 `eval_pipeline.md` 第四步。

完整工作流示例

```

1  # =====
2  # 第零阶段: ToMe 验证
3  # =====
4  cd ~/Uni-NaVid
5  git checkout main          # ToMe 改动提交到 main
6
7  # 修改 eva_vit.py 插入 ToMe, 提交
8  git add uninavid/model/multimodal_encoder/eva_vit.py
9  git commit -m "feat: add ToMe token merging to EVA ViT"
10
11 cd ~/NaVid-VLN-CE
12 bash eval.sh --model uninavid_baseline --benchmark r2r
13 bash eval.sh --model uninavid_baseline --benchmark r2r --use-tome
14 python compare_results.py \
15     --paths tmp/results_baseline_r2r tmp/results_baseline_tome_r2r
16
17
18 # =====
19 # 第一阶段: LLaMA-3.1-8B + 课程学习
20 # =====

```

```
21 cd ~/Uni-NaVid
22 git checkout -b feat/llama31
23 # 修改代码（见各阶段代码改动章节），提交后训练
24 bash scripts/uninavid_stage_1.sh
25 bash scripts/uninavid_stage_2.sh
26
27 cd ~/NaVid-VLN-CE
28 ln -s ~/Uni-NaVid/output/uninavid_s1_llama31_curriculum \
29     model_zoo/uninavid_s1_llama31_curriculum
30
31 git -C ~/Uni-NaVid checkout feat/llama31
32 bash eval.sh --model uninavid_s1_llama31_curriculum --benchmark r2r
33 python compare_results.py --tmp-dir tmp --sort sr
34
35
36 # =====
37 # 第二阶段: DPO（在 feat/llama31 分支继续）
38 # =====
39 cd ~/Uni-NaVid
40 # feat/llama31 分支上新增 train_dpo.py，准备三元组数据后训练
41 bash scripts/uninavid_dpo.sh
42
43 cd ~/NaVid-VLN-CE
44 ln -s ~/Uni-NaVid/output/uninavid_s2_dpo model_zoo/uninavid_s2_dpo
45
46 git -C ~/Uni-NaVid checkout feat/llama31
47 bash eval.sh --model uninavid_s2_dpo --benchmark r2r
48 python compare_results.py --tmp-dir tmp --sort sr
49
50
51 # =====
52 # 第三阶段: SigLIP（新建 feat/siglip 分支）
53 # =====
54 cd ~/Uni-NaVid
55 git checkout feat/llama31
56 git checkout -b feat/siglip
57 # 修改代码（siglip_encoder.py / builder.py / uninavid_arch.py），提交后训练
58 bash scripts/uninavid_stage_1_siglip.sh # 仅 Stage 1, 约 1-2 小时
59
60 cd ~/NaVid-VLN-CE
61 ln -s ~/Uni-NaVid/output/uninavid_s3_siglip model_zoo/uninavid_s3_siglip
62
63 git -C ~/Uni-NaVid checkout feat/siglip
64 bash eval.sh --model uninavid_s3_siglip --benchmark r2r
65 python compare_results.py --tmp-dir tmp --sort sr
```

磁盘空间参考

checkpoint	大小	代码分支	备注
<code>uninaavid_baseline</code> (Vicuna-7B, bf16)	~14 GB	<code>main</code>	已有
<code>uninaavid_s1_llama31_curriculum</code> (LLaMA-3.1-8B)	~15 GB	<code>feat/llama31</code>	第一阶段
<code>uninaavid_s2_dpo</code>	~15 GB	<code>feat/llama31</code>	第二阶段，与 S1 架构相同
<code>uninaavid_s3_siglip</code>	~12 GB	<code>feat/siglip</code>	第三阶段，视觉编码器更小
软链接	0 GB	—	推荐，节省磁盘

当前计划实际存储约 **56 GB**（全部软链接，原始文件在 `~/Uni-NaVid/output/` 下）。
若磁盘紧张，S1 checkpoint 在 S2 验收后可考虑删除（S2 是 S1 的超集）。